

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

***CO5:** Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks: 20 Duration: 60 Minutes Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I _____ A.Shiva Shankar Reddy (192425174) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ A.Shiva Shankar Reddy _____

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks (100)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Object Detection Pipeline

A system must: (A) Capture video frames, (B) Detect objects using a trained model, (C) Draw bounding boxes and labels, (D) Display results in real time. Constraints: process frames efficiently; handle multiple objects.

Pipeline: Real-Time Object Detection



Pseudocode

```
BEGIN RealTimeObjectDetection

// Module 1: Initialization
FUNCTION Initialize():
    model <- LoadTrainedModel(model_path)
    video <- OpenVideoCapture(source)
    IF video NOT opened THEN
        PRINT "Error: cannot access camera"
        EXIT
    ENDIF
    RETURN model, video
END FUNCTION

// Module 2: Frame Capture
FUNCTION CaptureFrame(video):
    success, frame <- video.Read()
    IF NOT success THEN
        RETURN NULL
    ENDIF
    RETURN frame
END FUNCTION

// Module 3: Object Detection
FUNCTION DetectObjects(model, frame):
    blob <- PreprocessFrame(frame)           // resize, normalize, scale
    rawDetections <- model.Forward(blob)     // single forward pass
    inference
    filtered <- []
    FOR EACH detection IN rawDetections DO
        IF detection.confidence > CONFIDENCE_THRESHOLD THEN
            APPEND detection TO filtered
        ENDIF
    ENDFOR
    finalResults <- NonMaxSuppression(filtered, IOU_THRESHOLD) //
    handles multiple/overlapping objects
    RETURN finalResults
END FUNCTION
```

```

// Module 4: Visualization
FUNCTION DrawResults(frame, results):
    FOR EACH (box, label, confidence) IN results DO
        DrawRectangle(frame, box, color=GREEN, thickness=2)
        text <- label + " " + FormatPercent(confidence)
        DrawText(frame, text, position=box.top_left)
    ENDFOR
    RETURN frame
END FUNCTION

// Module 5: Main Control Loop
FUNCTION Main():
    model, video <- Initialize()
    frameCount <- 0
    WHILE video.IsOpened() DO
        frame <- CaptureFrame(video)
        IF frame IS NULL THEN
            BREAK
        ENDIF
        frameCount <- frameCount + 1
        IF frameCount MOD SKIP_INTERVAL == 0 THEN           // process every
Nth frame for efficiency
            results <- DetectObjects(model, frame)
            frame <- DrawResults(frame, results)
        ENDIF
        Display(frame)
        IF KeyPressed('q') THEN
            BREAK
        ENDIF
    ENDWHILE
    video.Release()
    CloseAllWindows()
END FUNCTION

CALL Main()
END

```

Justification of Key Operations

Modular decomposition (Initialize, CaptureFrame, DetectObjects, DrawResults, Main) isolates each pipeline concern so the WHILE loop in Main stays lightweight and readable. The IF frameCount MOD SKIP_INTERVAL == 0 condition selectively skips inference on some frames, directly satisfying the 'process frames efficiently' constraint without dropping the display loop's real-time responsiveness. Inside DetectObjects, a confidence-threshold IF filter followed by NonMaxSuppression() removes duplicate/overlapping boxes around the same object, which is essential for correctly handling multiple simultaneously detected objects. The outer WHILE with a keypress BREAK condition provides a clean, controllable real-time display loop.

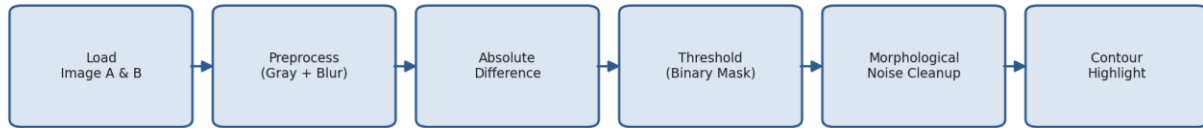
Constraint Satisfaction Summary

Constraint	How Pseudocode Satisfies It
Process frames efficiently	Frame-skip modulo check (SKIP_INTERVAL) and single-pass model inference reduce per-frame cost
Handle multiple objects	Loop over all detections + confidence filter + Non-Max Suppression correctly separates and retains each distinct object

2. Image Difference Detection System

A system must: (A) Read two images, (B) Compare them, (C) Highlight differences. Constraints: handle noise; ensure accurate difference detection.

Pipeline: Image Difference Detection



Pseudocode

```
BEGIN ImageDifferenceDetection

// Module 1: Image Loading
FUNCTION LoadImages(path1, path2):
    imgA <- ReadImage(path1)
    imgB <- ReadImage(path2)
    IF imgA IS NULL OR imgB IS NULL THEN
        PRINT "Error: could not read one or both images"
        EXIT
    ENDIF
    IF imgA.size <> imgB.size THEN
        imgB <- Resize(imgB, imgA.width, imgA.height)
    ENDIF
    RETURN imgA, imgB
END FUNCTION

// Module 2: Preprocessing
FUNCTION Preprocess(img):
    gray <- ConvertToGrayscale(img)
    denoised <- GaussianBlur(gray, kernel=(5,5))    // suppresses
sensor/compression noise
    RETURN denoised
END FUNCTION

// Module 3: Comparison Logic
FUNCTION ComputeDifference(grayA, grayB):
    diffMap <- AbsoluteDifference(grayA, grayB)
    _, binaryMask <- Threshold(diffMap, T=30, maxVal=255, method=BINARY)
    RETURN binaryMask
END FUNCTION

// Module 4: Noise Cleanup
FUNCTION CleanMask(mask):
    mask <- MorphologicalOpen(mask, kernel=(3,3))    // removes small
speckle noise
    mask <- MorphologicalClose(mask, kernel=(5,5))    // fills small gaps
in real changes
    RETURN mask
END FUNCTION
```

```

// Module 5: Visualization
FUNCTION HighlightDifferences(original, mask):
    contours <- FindContours(mask)
    FOR EACH contour IN contours DO
        IF ContourArea(contour) > MIN_AREA THEN      // ignore residual
noise blobs
            box <- BoundingBox(contour)
            DrawRectangle(original, box, color=RED, thickness=2)
        ENDIF
    ENDFOR
    RETURN original
END FUNCTION

// Module 6: Main
FUNCTION Main(path1, path2):
    imgA, imgB <- LoadImages(path1, path2)
    grayA <- Preprocess(imgA)
    grayB <- Preprocess(imgB)
    diffMask <- ComputeDifference(grayA, grayB)
    cleanMask <- CleanMask(diffMask)
    result <- HighlightDifferences(imgA, cleanMask)
    Display(result)
    SaveImage("difference_output.jpg", result)
END FUNCTION

CALL Main(image1_path, image2_path)
END

```

Justification of Key Operations

GaussianBlur() in the Preprocess module reduces pixel-level sensor noise before comparison, so trivial intensity fluctuations do not register as false differences — directly addressing the noise constraint. The Threshold() call converts the raw difference map into a clean binary mask, and the subsequent morphological open/close pair in CleanMask removes any remaining speckle noise while reconnecting fragmented regions of genuine change, which improves accuracy. The IF ContourArea(contour) > MIN_AREA condition in HighlightDifferences acts as a final safeguard, ensuring only differences of meaningful size are reported, which keeps the detection accurate rather than oversensitive.

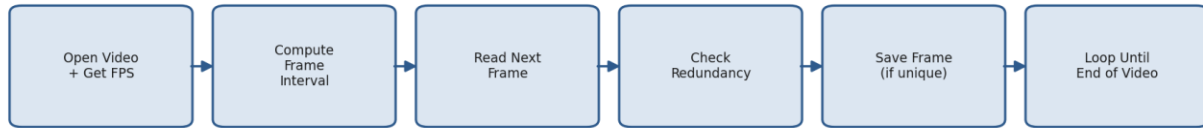
Constraint Satisfaction Summary

Constraint	How Pseudocode Satisfies It
Handle noise	GaussianBlur before differencing + morphological open/close after thresholding
Accurate difference detection	Absolute difference + binary threshold + minimum-area contour filter isolates only genuine, significant changes

3. Video Frame Extraction System

A system must: (A) Read video input, (B) Extract frames at fixed intervals, (C) Save selected frames. Constraints: efficient storage; avoid redundant frames.

Pipeline: Video Frame Extraction



Pseudocode

```
BEGIN VideoFrameExtraction

// Module 1: Initialization
FUNCTION Initialize(video_path, interval_seconds, output_dir):
    video <- OpenVideoCapture(video_path)
    IF video NOT opened THEN
        PRINT "Error: cannot open video"
        EXIT
    ENDIF
    fps <- video.GetFPS()
    frameInterval <- ROUND(fps * interval_seconds)    // frames to skip
between extractions
    CreateDirectoryIfNotExists(output_dir)
    RETURN video, frameInterval
END FUNCTION

// Module 2: Redundancy Check
FUNCTION IsRedundant(prevFrame, currFrame):
    IF prevFrame IS NULL THEN
        RETURN FALSE
    ENDIF
    grayPrev <- ConvertToGrayscale(prevFrame)
    grayCurr <- ConvertToGrayscale(currFrame)
    diffScore <- Mean(AbsoluteDifference(grayPrev, grayCurr))
    RETURN diffScore < REDUNDANCY_THRESHOLD          // near-identical
frame -> redundant
END FUNCTION

// Module 3: Frame Extraction and Saving
FUNCTION ExtractFrames(video, frameInterval, output_dir):
    frameCount <- 0
    savedCount <- 0
    lastSavedFrame <- NULL
    WHILE video.IsOpened() DO
        success, frame <- video.Read()
        IF NOT success THEN
            BREAK                                     // end of video
        ENDIF
        IF frameCount MOD frameInterval == 0 THEN
```

```

        IF NOT IsRedundant(lastSavedFrame, frame) THEN
            filename <- output_dir + "frame_" + savedCount + ".jpg"
            SaveImage(filename, frame, quality=COMPRESSION_QUALITY)
// compressed for efficient storage
            lastSavedFrame <- frame
            savedCount <- savedCount + 1
        ENDIF
    ENDIF
    frameCount <- frameCount + 1
ENDWHILE
video.Release()
RETURN savedCount
END FUNCTION

// Module 4: Main
FUNCTION Main():
    video, interval <- Initialize(video_path, N_SECONDS, output_dir)
    total <- ExtractFrames(video, interval, output_dir)
    PRINT "Total frames saved: ", total
END FUNCTION

CALL Main()
END

```

Justification of Key Operations

The frameInterval, computed from the video's FPS and the desired interval in seconds, drives the IF frameCount MOD frameInterval == 0 condition so frames are sampled at a fixed time interval rather than every frame, which is the core mechanism for efficient storage — far fewer images are written to disk than the raw frame count. The IsRedundant() module compares each interval-selected frame against the last saved frame using mean absolute difference; when the score falls below REDUNDANCY_THRESHOLD the frame is skipped, which prevents saving near-duplicate frames (e.g., during static scenes) and directly satisfies the 'avoid redundant frames' constraint. SaveImage() with a compression quality parameter further reduces the on-disk footprint of each retained frame.

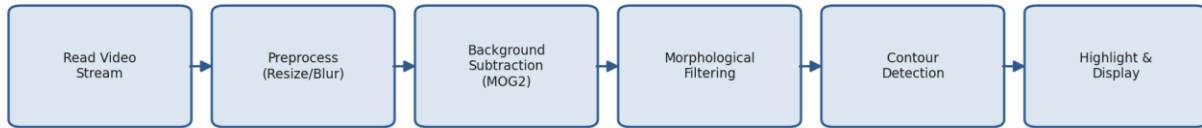
Constraint Satisfaction Summary

Constraint	How Pseudocode Satisfies It
Efficient storage	Fixed-interval sampling (MOD frameInterval) plus compressed image saving minimizes the number and size of stored files
Avoid redundant frames	IsRedundant() mean-difference check skips saving frames nearly identical to the last saved frame

4. Background Subtraction Pipeline

A surveillance system must: (A) Read video stream, (B) Separate foreground from background, (C) Highlight moving objects. Constraints: handle dynamic background; reduce noise.

Pipeline: Background Subtraction



Pseudocode

```
BEGIN BackgroundSubtractionPipeline

// Module 1: Model Initialization
FUNCTION InitializeModel():
    bgSubtractor <- CreateBackgroundSubtractorMOG2(history=500,
varThreshold=16, detectShadows=TRUE)
    RETURN bgSubtractor // adaptive Gaussian-mixture model
tolerates gradual background change
END FUNCTION

// Module 2: Preprocessing
FUNCTION Preprocess(frame):
    resized <- Resize(frame, WIDTH, HEIGHT)
    smoothed <- GaussianBlur(resized, kernel=(5,5)) // reduces sensor
noise before modeling
    RETURN smoothed
END FUNCTION

// Module 3: Background Subtraction
FUNCTION SubtractBackground(bgSubtractor, frame):
    fgMask <- bgSubtractor.Apply(frame, learningRate=ADAPTIVE_RATE) //
continuously updates background model
    RETURN fgMask
END FUNCTION

// Module 4: Mask Filtering
FUNCTION FilterNoise(mask):
    mask <- MorphologicalOpen(mask, kernel=(3,3)) // removes small
noise specks
    mask <- MorphologicalClose(mask, kernel=(7,7)) // fills holes
inside real objects
    mask <- MedianBlur(mask, 5) // extra salt-and-
pepper noise reduction
    RETURN mask
END FUNCTION

// Module 5: Object Highlighting
FUNCTION HighlightObjects(frame, mask):
    contours <- FindContours(mask)
```

```

    FOR EACH contour IN contours DO
        IF ContourArea(contour) > MIN_AREA THEN
            box <- BoundingRect(contour)
            DrawRectangle(frame, box, color=YELLOW, thickness=2)
        ENDIF
    ENDFOR
    RETURN frame
END FUNCTION

// Module 6: Main Loop
FUNCTION Main(video_path):
    video <- OpenVideoCapture(video_path)
    bgSubtractor <- InitializeModel()
    WHILE video.IsOpened() DO
        success, frame <- video.Read()
        IF NOT success THEN
            BREAK
        ENDIF
        pre <- Preprocess(frame)
        mask <- SubtractBackground(bgSubtractor, pre)
        cleanMask <- FilterNoise(mask)
        output <- HighlightObjects(pre, cleanMask)
        Display(output)
        IF KeyPressed('q') THEN
            BREAK
        ENDIF
    ENDWHILE
    video.Release()
END FUNCTION

CALL Main(video_path)
END

```

Justification of Key Operations

CreateBackgroundSubtractorMOG2() maintains a per-pixel Gaussian mixture model that is updated every frame via the learningRate parameter in SubtractBackground, allowing the model to continuously adapt to slow scene changes such as moving shadows, swaying trees, or gradual illumination shifts — this is what satisfies the dynamic-background constraint, as opposed to a single static reference frame. The FilterNoise module chains morphological opening, closing, and a median blur specifically to reduce noise: opening removes small false-positive specks in the foreground mask, closing fills small holes inside genuine moving objects, and the median blur removes residual isolated pixel noise. The final IF ContourArea(contour) > MIN_AREA check discards any remaining insignificant blobs before drawing, ensuring only real moving objects are highlighted.

Constraint Satisfaction Summary

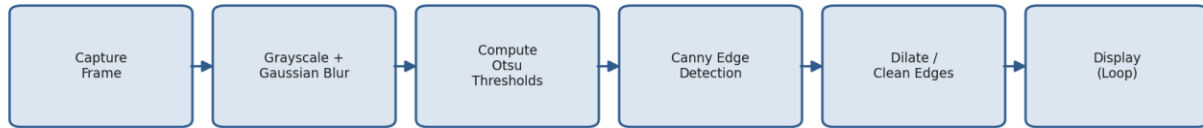
Constraint	How Pseudocode Satisfies It
Handle dynamic background	MOG2 adaptive Gaussian-mixture model with continuous learningRate updates absorbs gradual background changes
Reduce noise	Gaussian blur in preprocessing + morphological open/close + median blur + minimum-area contour filter

5. Edge Detection Pipeline for Real-Time Video

A system must: (A) Capture video frames, (B) Apply edge detection, (C) Display results.

Constraints: real-time processing; maintain clarity of edges.

Pipeline: Real-Time Edge Detection



Pseudocode

```
BEGIN EdgeDetectionPipeline

// Module 1: Initialization
FUNCTION Initialize(source):
    video <- OpenVideoCapture(source)
    IF video NOT opened THEN
        PRINT "Error: cannot access video source"
        EXIT
    ENDIF
    RETURN video
END FUNCTION

// Module 2: Preprocessing
FUNCTION Preprocess(frame):
    gray <- ConvertToGrayscale(frame)           // reduces channels ->
    faster processing
    blurred <- GaussianBlur(gray, kernel=(5,5)) // suppresses noise that
    fragments edges
    RETURN blurred
END FUNCTION

// Module 3: Adaptive Threshold Computation
FUNCTION ComputeThresholds(frame):
    medianVal <- Median(frame)
    lowThresh <- MAX(0, 0.66 * medianVal)
    highThresh <- MIN(255, 1.33 * medianVal)
    RETURN lowThresh, highThresh
END FUNCTION

// Module 4: Edge Detection
FUNCTION DetectEdges(frame, lowThresh, highThresh):
    edges <- CannyEdgeDetector(frame, lowThresh, highThresh) // detects
    strong, continuous edges
    RETURN edges
END FUNCTION

// Module 5: Edge Refinement
FUNCTION RefineEdges(edges):
```

```

        refined <- Dilate(edges, kernel=(2,2), iterations=1)    // strengthens
thin/broken edges for clarity
        RETURN refined
    END FUNCTION

// Module 6: Main Real-Time Loop
FUNCTION Main(source):
    video <- Initialize(source)
    WHILE video.IsOpened() DO
        success, frame <- video.Read()
        IF NOT success THEN
            BREAK
        ENDIF
        pre <- Preprocess(frame)
        lowT, highT <- ComputeThresholds(pre)
        edges <- DetectEdges(pre, lowT, highT)
        finalEdges <- RefineEdges(edges)
        Display(finalEdges)
        IF KeyPressed('q') THEN
            BREAK
        ENDIF
    ENDWHILE
    video.Release()
    CloseAllWindows()
END FUNCTION

CALL Main(source)
END

```

Justification of Key Operations

Converting to grayscale in Preprocess reduces the data the Canny detector must process from three channels to one, which is essential for maintaining real-time throughput.

ComputeThresholds derives the Canny low/high thresholds automatically from the frame's median intensity rather than using fixed constants, so edge sensitivity adapts frame-to-frame to changing scene brightness while still using a single fast function call, preserving real-time performance. GaussianBlur before detection removes noise that would otherwise fragment edges into disconnected segments, and the Dilate() step in RefineEdges thickens and reconnects thin or slightly broken edge lines — both operations directly serve the 'maintain clarity of edges' constraint. The WHILE loop with a keypress exit condition keeps the pipeline running continuously frame-by-frame for real-time display.

Constraint Satisfaction Summary

Constraint	How Pseudocode Satisfies It
Real-time processing	Single-channel grayscale conversion, one Canny call per frame, and a lightweight WHILE loop keep per-frame cost low
Maintain clarity of edges	Noise-reducing blur before detection, adaptive median-based thresholds, and post-detection dilation to reconnect/strengthen edges